

gt security guide — gt unlock and gt sandbox mode

Written against **gt v0.20.0** (gt-lotr 0.3.0). Unlock and sandbox mode both ship **off**. This guide says what each does when you turn it on, what it needs, how to set it up on macOS, Windows and Linux, and — just as plainly — what it does not protect. gt sandbox mode, which fences the assistant's own shell and file tools off the vault and gt's state, is section 8.

"Unlock proves a person was present and limits what agents can do on their own. It is not anti-malware. If something already runs as you, it can wait for you to unlock."

1. The goal, and what was done to get there

gt aims to make everything it touches as secure as it can, and to tell you exactly how secure that is on your machine. An AI agent working for you runs as you: it can call tools, run shell commands and read your files. gt unlock puts a person back in the loop for the things that matter — your connected systems (LOTR), your credentials, publishing, and gt's own guards — without slowing down ordinary work.

Read this first. In 0.20.0 the authority runs **as you** (levels L1 and L2). Against a program running as you — and that includes the assistant's own shell — the gate is **friction**: it stops accidents and makes the assistant ask, but a determined process of yours can get around it (section 4 lists how). The guarantees that hold even then are narrower and come from hardware: at **L2**, opening a sealed credential needs your finger or PIN every time (or once per short window you choose), and a consent-tier LOTR operation can need a touch of its own. A real boundary against the agent needs **L3** — the authority as a separate, administrator-owned service — which **0.20.0 does not ship**.

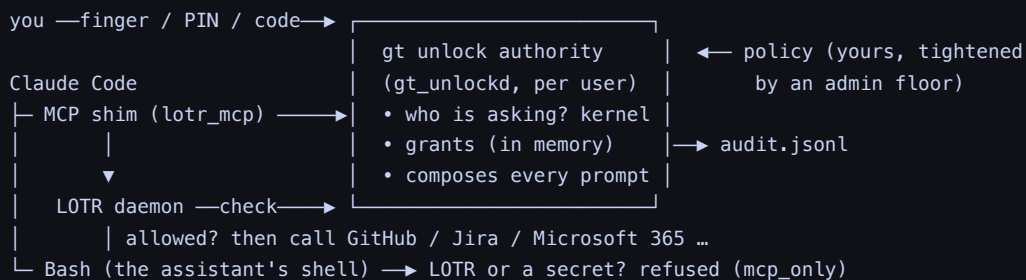
An independent security review of 0.20.0 (2026-10-03) reproduced seven bypasses of the gate by a same-user process. Each was fixed, and each is now a regression test (`tests/test_unlock_review_regressions.py` , `tests/test_lotr_review_regressions.py`); what cannot be fixed without L3 is written down in section 4 instead of being claimed away. A second, fresh-context review (2026-10-04) found more: the sandbox's inbox could be raced into copying a read-denied file (the TOTP seed) into the vault, an orphan could give itself a terminal and read without unlocking, an older signed policy approval could be replayed, a repository's own settings could loosen the sandbox unreported, the file tools could plant code where Claude Code or your shell loads it, and requests could name a session they do not belong to. Each is fixed or, where it cannot be prevented at L1/L2, reported and written down here; each demonstrated bypass is a regression test (`tests/test_inbox_review2.py` , `tests/test_sandbox_review2.py` , `tests/test_unlock_review2_regressions.py`).

What was built, and what each piece really does:

MEASURE	WHAT IT DOES	AGAINST A PROCESS RUNNING AS YOU (L1/L2)
Kernel peer identity. Every caller is identified by the operating system (macOS audit token with the process version, Linux <code>S0_PEERCRED</code> + start time, Windows named-pipe client pid + creation time + user SID) — never by anything the caller says	A process cannot pretend to be another live process	Holds
The client checks the server. Before sending anything, gt's client asks the kernel which process serves the authority's address and refuses it unless it runs the installed <code>gt_unlockd.py</code>	A fake "allow everything" server bound after the real one stopped is refused	Friction: a process of yours can run the <i>real</i> <code>gt_unlockd.py</code> with its own state
Signatures, not booleans. Touch ID and Windows Hello sign a fresh challenge with a hardware key (Secure Enclave / TPM); gt verifies the signature itself in Python	A fake helper that answers "ok"; a replayed old approval	Holds for presence. Before each use the helper is read once, hashed against what install recorded, and run from a private copy of exactly those bytes, so a swap after the check runs nothing
Sealed credentials (<code>sealed:</code> refs). Encrypted at rest under a biometric hardware key; every open needs your finger or PIN (or once per <code>secrets_window_s</code> , per process and grant, never shared)	Credential theft from disk; reading a sealed value without you touching the sensor	Holds at L2. Once opened, the value is in the requesting program's memory
Biometric consent (<code>consent_requires_factor:</code> <code>platform</code>). A touch over each consent-tier operation, composed by the authority from the operation itself	A screen-control agent clicking "approve"	Holds at L2 , as long as you read what the prompt names
The <code>mcp_only</code> door. LOTR and the credential broker serve only the MCP connection Claude Code started (the installed <code>lotr_mcp.py</code> , child of <code>claude</code>) — not a command run in the assistant's shell	The assistant using LOTR or your secrets through Bash by accident or by habit	Friction (section 4)
Only the authority asks. Every prompt is composed by gt's unlock authority, naming who is asking and for what; a caller's own reason appears only as a quoted, unverified claim. A client can request an unlock; it can never claim one	An agent typing "approved" on your behalf	Holds; a process can still show a look-alike prompt of its own
Grants in memory only , bound to one session and one seat (the session's vault MCP server holds its own grant, apart from LOTR's shim and the shell, so approving one does not unlock the other) and revoked on idle (15 min), TTL (8 h), screen lock, sleep, clock rollback, the session or MCP shim ending (a replacement shim starts with nothing), and <code>gt_unlock.py lock</code>	A forgotten unlocked session staying open	Holds for timing; see section 4 for who can use a live grant
Policy approval signed by your platform factor. A policy edited behind gt's back is not trusted until approved with a step-up, and on macOS /	Rewriting <code>policy.json</code> and	Friction: a process of yours can rewrite your enrolment too (section 4). TOTP-only (Linux): a hash, L1

MEASURE	WHAT IT DOES	AGAINST A PROCESS RUNNING AS YOU (L1/L2)
Windows the approval is a Touch ID / Hello signature over the policy itself, a monotonic serial and its time; the authority keeps the highest serial it has accepted, so an older signed approval restored with its policy fails closed	its approval to open everything	
Admin policy floor. An administrator-owned file that can only tighten; each scope's level is the stricter of yours and the admin's, each by its own most specific rule	A user pattern loosening an organisation's rule	Holds as logic; binding only at L3
Fail closed. With unlock on, an unreadable or missing policy, deleted policy files, a missing factor, an authority that is down or cannot be verified, an unwritable audit log (for writes and secrets), or a policy edited behind gt's back locks every gated action — never the reverse	"Broken" quietly meaning "open"	Friction: deleting the <code>unlock-on</code> markers too turns it off
No gt crypto store. gt brokers proven stores (1Password, sops+age, Vault, the OS keychains) and uses CryptoKit / DPAPI for sealing; it ships no cipher of its own	Home-made cryptography	Holds
gt sandbox mode (section 8, separate from unlock). Claude Code's own OS sandbox (Seatbelt / bubblewrap) around the assistant's shell, plus permission rules on its file tools, keep both off the vault, gt's state, the unlock home and LOTR, and the file tools off every file Claude Code, your shell or launchd loads code from; the vault is reached through gt's MCP server and write queue	The assistant's shell reading the TOTP seed, rewriting gt's policy, enrolment, hooks or plugin files, or writing the vault directly	Holds on macOS, Linux and WSL2 for the shell and its children (an OS boundary) while nothing loosens it: a repository's own <code>.claude/settings*.json</code> can, unless the sandbox is admin-required through managed settings or <code>claude --settings</code> (section 8.4); <code>gt_sandbox.py check</code> reports every loosening key. Native Windows: permission rules only — friction
Every grant audited. Each unlock, check, secret read and revocation is one line in <code>~/.claude/golden-thread/unlock/audit.jsonl</code> , with its grant id — never a code or a secret value. A write, consent or secret release that cannot be recorded is refused	"What happened while I was unlocked?"	The log is yours: a process of yours can edit it afterwards

2. How it works



1. A session starts. `gt` registers it with the authority; the LOTR MCP connection Claude Code starts registers itself as that session's one shim. The authority accepts it only when the kernel says its parent is `claude` **and** the process runs the installed `gt-lotr lotr_mcp.py` (by real path, from Claude Code's `installed_plugins.json`) under an isolated interpreter (`python -I`, below). A second shim is refused while the first is alive; a shim that replaces a dead one starts with no grant. If the authority restarts, the shim registers again on its next call. On Windows the installer points the MCP command at the Python interpreter itself rather than at a launcher script (0.20.0): a script in between would be the parent, and every registration would be refused.
2. The assistant calls a LOTR tool. LOTR asks the authority whether **this process** (named by the kernel) may use `lotr:<connection>:<tier>`. Locked: the authority asks you — a Touch ID or Windows Hello sheet, then a dialog for your authenticator code — naming the requester and the scope.
3. You approve. The authority checks the signature and the code, grants the session, and LOTR proceeds. The grant id is written into LOTR's audit line for that call.
4. A sealed credential the call needs is opened by the authority with **another** touch (every time by default; `gt_unlock.py policy secrets-window 60` allows one touch per minute per process and grant). Only `gt-lotr`'s installed `lotrd.py` may ask for a credential on behalf of the shim, and the value goes to the downstream call, never back to the assistant.
5. Fifteen idle minutes, eight hours, your screen locking, the lid closing, or the session ending: the grant is gone and the next call asks again.

Reads of LOTR connections can stay open (`read_without_unlock`, on by default) — for a person at a terminal. The requesting process must have a controlling terminal **and** a login shell above it (`argv[0]` starting with `-`, or `-l` / `--login` — what `login(1)`, `sshd`, `Terminal`, `iTerm2` and `tmux` start). An orphan (re-parented to `launchd` or `init`, no login shell above it) never gets it, even after giving itself a terminal with `setsid + openpty + TIOCSCTTY` (the 2026-10-04 re-review did exactly that against 0.20.0's first build; it is now a regression test). On Windows the parent chain must be intact and reach an interactive shell or terminal host. This is a convenience, not a boundary: a process of yours can start its own login shell (`bash -lc`, `script`, a scripted `Terminal` window). For a boundary set `read_without_unlock false`. Terminals that start non-login shells (GNOME `Terminal`'s default) need an unlock for these reads. Writes and anything gated need the grant. Consent-tier operations (`send`, `merge`, `delete`) still get LOTR's own confirmation, and you can require a Touch ID / Hello approval per operation instead (`gt_unlock.py policy consent platform --window 300` approves consent operations for five minutes, then asks again). Stopping the authority while unlock is on needs a fresh factor; `gt_unlock.py lock` never does.

3. What it needs, and how to set it up

Everything below runs from your own terminal. `gt_unlock.py` lives at `~/.claude/golden-thread/hooks/gt_unlock.py` (written `gt_unlock.py` below).

MACOS — TOTP + TOUCH ID (L2)

Needs: a Mac with Touch ID, the Xcode Command Line Tools (`xcode-select --install`, so `install.sh` can build the Touch ID helper), and an authenticator app on your phone.

1. Re-run `install.sh`. It builds `~/.claude/golden-thread/bin/gt-presence` from the shipped Swift source and ad-hoc signs it (a Developer ID signed build is provided with releases).
2. `gt_unlock.py enroll touchid` — touch the sensor once. This creates two Secure Enclave keys that only work with your current fingerprints (adding or removing a fingerprint retires them, by design).
3. `gt_unlock.py enroll totp` — scan the QR code, type the code your app shows. The screen is cleared afterwards; the seed is shown only now.
4. `gt_unlock.py recovery` — print ten one-time recovery codes and store them offline.
5. `gt_unlock.py policy enable` — proves your factors work (touch + code), then turns unlock on.
6. `gt_unlock.py status` should say `level L2`. `gt_unlock.py verify` runs the self-check.

WINDOWS — TOTP + WINDOWS HELLO (L2)

Needs: Windows 10/11 with Windows Hello set up (Settings > Accounts > Sign-in options > PIN (Windows Hello); a fingerprint or face reader is optional), and an authenticator app.

1. `gt_unlock.py enroll hello` — Windows Security asks for your PIN or biometric a few times (key creation, a check that the key signs deterministically, which sealing relies on, and a final proof). The Hello dialog shows its own wording ("Making sure it's

you"); gt shows the requester and scope in its own prompt.

2. `gt_unlock.py enroll totp` , `gt_unlock.py recovery` , `gt_unlock.py policy enable` as above.

The Hello helper is a Windows PowerShell 5.1 script run with `-ExecutionPolicy Bypass` for its own process only; nothing about your machine's execution policy changes. It is not code-signed: the security rests on the TPM signature gt verifies, not on who signed the script. Organisations that block unsigned scripts sign it with their own certificate. Sealing on Windows needs a desktop logon (user-scope DPAPI is unavailable in an SSH key logon).

LINUX — TOTP (L1), OPTIONALLY ENTRA

Linux has no platform authenticator gt can use, so unlock there is **L1**: it stops agents and accidents, not malware running as you.

`gt_unlock.py enroll totp` , `gt_unlock.py recovery` , `gt_unlock.py policy enable` . Without a desktop, codes are typed in the terminal that asked.

MICROSOFT ENTRA ID SIGN-IN (ANY PLATFORM, OPTIONAL)

A fresh browser sign-in (`prompt=login`) can be one of your factors: authorization code + PKCE on a `http://localhost` redirect, the ID token's signature and claims checked by gt (issuer, tenant, audience, nonce bound to the request, sign-in time). You register an app once:

1. Entra admin center > App registrations > New registration. Account types: your organisation only (strongest), any organisation (`organizations`), or work and personal accounts (`common`).
2. Redirect URI: platform *Public client/native (mobile & desktop)*, value `http://localhost` .
3. No client secret. API permissions: Microsoft Graph delegated `openid` , `profile` , `email` only. Leave *Allow public client flows* off.
4. Record the Application (client) ID and Directory (tenant) ID, then: `gt_unlock.py policy sso --client-id <app id> --tenant <tenant id|organizations|common>` and `gt_unlock.py enroll sso` .
5. Enforce MFA in Entra with Conditional Access (authentication strength) on the app; v2 ID tokens do not always say how you signed in. Sign in with Apple is not supported (it cannot redirect to a local address and carries no tenant or MFA claims).

4. What each level protects — and what it does not

`gt_unlock.py status` and `/gt:gt-doctor` (rows `unlock` and `security`) name the level this machine runs at. Nothing here says "secure"; each check passes, fails, or says it could not be checked.

LEVEL	WHAT IT IS	STOPS	DOES NOT STOP
off (default)	Nothing gated	—	Anything
L1 Gate	TOTP (and SSO), no platform factor	Accidents, use after the session ends or the screen locks, the assistant using LOTR or secrets <i>by the ordinary route</i> ; full audit. Friction, not a boundary , against the assistant's shell	Any program running as you — the assistant's shell included — that sets out to get around it: it can read the TOTP seed and answer the code itself
L2 Sealed	Touch ID / Windows Hello required in every unlock; credentials sealed	Reading a sealed credential without your finger or PIN (each open, or each short window); a consent-tier operation without a touch, when you require one; screen-control agents clicking approve (they cannot touch the sensor). Everything else as L1 — still friction against a process running as you	A program running as you using what you approve: it can ask while you are there, race a look-alike prompt, use an unlocked grant, or get around the gate itself (below); code patched to capture your next unlock
L3 Separated	Authority and LOTR as a separate service account, code and policy administrator-owned	Use before unlock, policy downgrade, seed and credential reads, the gate bypasses below — the agent boundary	An attacker inside your session after you unlock. gt 0.20.0 does not ship L3.

THREAT	L1	L2
The assistant using LOTR or secrets from Bash <i>by the ordinary route</i>	Refused (<code>mcp_only</code>)	Refused
The assistant's shell, or any program of yours, setting out to get around the gate	Not stopped — friction only (list below)	Not stopped for the gate; sealed values still need your touch per open
Another local agent or script, while locked	Gated, but can read unsealed credentials directly	Sealed credentials stay sealed
Malware running as you, before you unlock	Not stopped	Partly: it cannot open sealed credentials without your touch, but can change gt's code for your next unlock
Malware running as you, after you unlock	Not stopped	Not stopped for the gate; each sealed open still needs a touch, which it can ask you for
A screen-control / accessibility agent	Can click dialogs	Cannot touch the sensor
A keylogger reading your TOTP code	First use wins (each code works once)	Same; use TOTP with a platform factor
A fake prompt racing gt's	Possible	Possible: gt's prompts are composed by the authority and name the requester and scope — read them
Stolen laptop, locked	Disk encryption + revoke on lock	The Secure Enclave / TPM needs your finger or PIN
Stolen laptop, unlocked and unattended	Until idle / TTL (15 min idle by default)	Same, but each sealed open needs a finger
Identity provider compromise	SSO forged; two factors including a local one mitigate	Same
Root / administrator compromise	Out of scope	Out of scope

How a program running as you can still get around the gate at L1/L2 — each is friction, documented, and closed only by L3. With **gt sandbox mode** on (section 8, macOS / Linux / WSL2), the assistant's *shell* is fenced off the files most of these routes need — the unlock home, gt's hooks and state, the plugin files, LOTR — but every other program of yours is not:

- **Run its own authority.** It can kill the daemon (a signal needs no factor) and start the *real* `gt_unlockd.py` against its own home at the same address; gt's clients check the program, not whose state it serves. A restart revokes every grant, so this shows up as a fresh prompt or a refusal, not silent access to yours.
- **Rewrite your files.** The policy, its approval, the enrolment, `installed_plugins.json` and gt's own code are your files. The policy approval is a Touch ID / Hello signature, but a process of yours can create its own Secure Enclave key, write it into the enrolment and sign with it (a Secure Enclave key is not bound to the program that made it). An approval by a key the enrolment marks as needing no finger is refused — the marking itself is your file. Deleting the policy and its state fails closed while the `unlock-on` markers exist; deleting the markers too turns unlock off. Restoring an *older* signed policy with its `state.json` does not work: the approval signs a serial and its time, and the authority refuses a serial below the highest it has accepted (kept in memory and in `approval-floor.json`). Approvals made before this check count as serial 0 until the first new one. Rewriting the floor file and restarting the authority rolls it back; while the authority runs, the in-memory floor holds.
- **Read the TOTP seed** (`totp.seed`). At L1 that is the whole unlock: it can compute a code and unlock its own session — and, being in the same session as the shim, the shim's.

- **Look like gt's own processes.** The shim seat and the credential consumer are checked by the real path of the file a process runs; a process of yours can run the real file with its own input. A replacement shim starts with no grant, and the first shim keeps its seat while it lives. The real file can also be made to run someone else's code: before 0.20.0's re-review fix, `PYTHONPATH` pointing at a `sitecustomize.py` ran arbitrary code inside the real file, under its trusted identity. Now every trusted Python process (`lotrd.py` , `lotr_mcp.py` , `gt_vault_mcp.py` , `gt_unlockd.py`) must also run under an isolated interpreter: gt starts each with `python -I` , and the authority and its clients refuse a peer whose command line lacks `-I` (or both `-E` and `-s`), or cannot be read. On macOS and Linux a refusal names the injection variables it saw, never their values; on Windows the environment is not read and only the flag check applies. **Start a hand-run lotrd with `python3 -I lotrd.py`** ... — one started without it is no longer trusted while unlock is on. Not covered: a process of yours that modifies the interpreter or its site-packages (Homebrew's are writable by you).
- **Leave the session.** A double-forked process is not a descendant of `claude` , so the `mcp_only` door does not apply to it; it gets no read without unlock, and any unlock it asks for is a prompt naming it — even if it gives itself a terminal (above). On Windows, where there is no controlling terminal, its broken parent chain is what refuses it.
- **Claim a scheduled job's name** (`GT_JOB`) from outside a session and use that job's allow-listed, narrow, read-only scope. List only what you would accept that for.
- **Swap the Touch ID helper.** The helper is hashed at install. Each use reads it once through a no-follow descriptor, hashes exactly those bytes against the install record, writes them to a fresh private 0700 folder, checks the code-signing requirement on that copy, runs the copy and deletes it, so swapping the helper between the check and the run runs nothing. But the record is your own file: a process that rewrites the helper *and* its install record can watch what is sealed or opened afterwards (not forge your presence; a release binary's Developer ID signature is checked too), and a process of yours could in principle race the private copy.
- **Edit the audit log** after the fact, and the hooks' settings guard sees only Claude Code's Write/Edit tools, not a shell command writing `settings.json` .

Other honest limits:

- A brokered secret is only as locked as the weakest of the store, gt's grant, and the way it reaches the program. Environment variables are readable by any process of yours (`ps eww`). Only `sealed:` refs need your touch per open; `file:` , `keychain:` (L1) and the other stores are as strong as their own table row in section 5.4.
- The admin floor and the policy checks stop agents and accidents, not someone with your account; on a Mac where the daily account is an administrator, even L3's admin-owned files are bounded by the administrator password prompt.

5. Locking down more with gt unlock

5.1 THE VAULT: LOCK CHOSEN FOLDERS, NOT THE VAULT

`gt_lock.py add Secrets/aws.md --vault <vault>` encrypts a file with `age` to `aws.md.age` and removes the plaintext; a `.gt-locked` note in the folder says what is locked and how to open it. `gt_lock.py open Secrets/aws.md.age --vault <vault> | less` decrypts to a pipe only (a terminal needs `--show` ; a file is always refused) after a `gt:lock:<folder>` grant; `gt_lock.py restore` puts the plaintext back (mode 600); `gt_lock.py status` lists what is locked and how strong each lock is. The identity comes from `--identity` , `$GT_AGE_IDENTITY` , `SOPS_AGE_KEY_FILE` , or `~/.config/sops/age/keys.txt` . Lock before the first commit or sync: plaintext already in git history, sync version history or backups stays there.

- **Strength:** L2 only when every recipient is hardware-held (`age1se1...` Secure Enclave via `age-plugin-se` , `age1yubikey1...`); L1 when any recipient or the identity is a plaintext key such as `~/.config/sops/age/keys.txt` — the usual setup. Never L3: once opened, any process of yours can read the plaintext.
- `core-rules/` and `global-memory/` can never be locked (enforced in code): gt reads them on every turn, and locking them would silently switch Core-rule injection off.
- Why not the whole vault: Obsidian keeps it open, gt's hooks read it on every prompt, and an encrypted volume synced by Dropbox and mounted on two machines can corrupt.
- What others see: Obsidian shows links into a locked file as unresolved; Dropbox, iCloud and git see only `.age` ciphertext (commit only ciphertext); gt's lint, tasks and recall treat a locked folder as absent and say so once.
- An encrypted disk image (macOS sparse bundle) or BitLocker VHDX is a bring-your-own option; once mounted, any process of yours can read it.

5.2 OTHER APPS' CREDENTIALS, THROUGH THE BROKER

Credentials are released only under a grant, and handed over by pipe or a short-lived 600 file, never by environment variable unless you ask for it.

- **git / GitHub publishing:** store the token sealed (`gt_unlock.py seal put github` , value on stdin), map the host in `~/.claude/golden-thread/unlock/credentials.json` (`{"git": [{"protocol": "https", "host": "github.com", "username": "x-access-token", "ref": "sealed:github"}]}`), and set `git config --global credential.https://github.com.helper "!python3 ~/.claude/golden-thread/hooks/gt_unlock.py git-credential"` . Every push then needs a `gt:publish` grant. It is real only if the token exists nowhere else (no plaintext `~/.config/gh/hosts.yml` , no unencrypted SSH key with the same reach).
- **AWS:** `credential_process = python3 ~/.claude/golden-thread/hooks/gt_unlock.py aws-credential --ref sealed:aws` (the sealed value is the JSON AWS expects).
- **Any command:** `gt_unlock.py run --scope gt:publish --secret-file GH_TOKEN_FILE=sealed:github -- <cmd>` hands the command a path to a 600 file deleted when it exits. `--env VAR=REF` puts the value in the environment instead, which is weaker, and `gt` says so each time.
- `kubectrl exec` plugins and `docker` credential helpers can call `gt_unlock.py secret get REF` the same way (it refuses to print to a terminal).
- Moving an existing credential in: `gt_unlock.py seal migrate file:/abs/path --name github --remove-source` (also `store:` and `keychain:`).

5.3 SSH KEYS

Keep SSH keys in hardware and use them per signature: Secure Enclave keys through an agent such as `Secretive` (macOS) or a FIDO2 security key (`ssh-keygen -t ed25519-sk`). `gt` does not hold SSH keys; an unencrypted key file in `~/.ssh` is readable by any process of yours whatever `gt` does.

5.4 BRING YOUR OWN SECRET STORE

Refs `sops:` , `op:` , `bw:` , `vault:` and `wincred:` are resolved by the authority under a grant, and `gt_unlock.py status /` doctor label each with its real level:

STORE	LEVEL AGAINST PROCESSES RUNNING AS YOU
Keychain via <code>/usr/bin/security</code> (LOTR's <code>keychain:</code> refs)	L1
Windows Credential Manager / DPAPI (<code>wincred:</code>)	L1
sops + age, identity in a plaintext file	L1
sops + age, identity in the Secure Enclave or a YubiKey	L2
1Password CLI (<code>op:</code>)	L2 (its own biometric unlock and idle limit)
Bitwarden CLI (<code>bw:</code>)	L1 after unlock (the session key sits in the environment or a file)
HashiCorp Vault (<code>vault:</code>)	L1 (0.20.0 does not hold Vault tokens in the authority)
gt's sealed store (<code>sealed:</code>)	L2

5.5 SCHEDULED JOBS

Nothing is allowed unattended by default. Each job may be given one narrow scope: `gt_unlock.py policy unattended add <job> lotr:<connection>:read` (or `secret:<ref>` for one credential), and the job's definition sets `GT_JOB=<job>` . Write, consent, publish, wildcard and `gt`'s own scopes can never be allowed unattended; jobs are never prompted.

5.6 WHAT GT UNLOCK CANNOT LOCK

- Applications that never ask `gt`: a browser, an editor, any program reading its own config.

- Anything after you unlock, inside the unlocked window, from software already running as you.
- Credentials that also exist somewhere else in plaintext.
- Files outside the folders you lock.

WORKED EXAMPLE (MACOS)

```
gt_unlock.py enroll touchid && gt_unlock.py enroll totp && gt_unlock.py recovery
gt_unlock.py policy enable
printf %s "$GITHUB_TOKEN" | gt_unlock.py seal put github      # then delete the plaintext copy
lotr --home ~/.config/gt-lotr/personal add-http github@personal --profile github \
    --base-url https://api.github.com --identity me --auth bearer --token-ref sealed:github
gt_lock.py add Secrets/aws.md --vault ~/Vault
gt_unlock.py verify
```

WORKED EXAMPLE (WINDOWS, POWERSHELL)

```
$u = "$env:USERPROFILE\.claude\golden-thread\hooks\gt_unlock.py"
python $u enroll hello; python $u enroll totp; python $u recovery
python $u policy enable
python $u seal put github      # type the token at the hidden prompt
python $u verify
```

6. Enterprise

- **Admin policy floor:** `/Library/Application Support/gt/unlock-policy.json` (macOS, root-owned with root-owned parents, or delivered by an MDM profile that writes it), `%ProgramData%\gt\unlock-policy.json` (Windows, owned by Administrators/SYSTEM with no user write access) or `HKLM\SOFTWARE\Policies\gt\UnlockPolicy` (a JSON string, for GPO/Intune), `/etc/gt/unlock-policy.json` (Linux). Its existence turns unlock on. It only tightens: more factors, fewer allowed factors, shorter TTL and idle, stricter scopes, `locked_keys` the user cannot change, a pinned Entra tenant. A floor file `gt` cannot verify locks everything.
- The floor is **binding only at L3**, when the enforcing code is also administrator-owned; 0.20.0 runs the authority as the user. Say so in your own policy documents. Where users are administrators of their own machines, admin-owned protection is bounded by that password.
- **Signing:** release builds of the macOS helper are Developer ID signed and notarized. The Windows Hello script is unsigned by design; sign it with your own certificate where policy requires signed scripts.
- **Recovery for managed fleets:** keep `admin_only` recovery in your process: reset an enrolment by removing `~/.claude/golden-thread/unlock/` and the marker beside it, `~/.claude/golden-thread/.unlock-unlock-on` (which also discards sealed credentials), under your own controls.

7. Recovery, and turning it off

- **Lost phone or changed fingerprints:** `gt_unlock.py unlock --recovery` accepts one recovery code as one factor; you then re-enrol (`gt_unlock.py enroll ...`). Each code works once.
- **Lost every factor:** remove `~/.claude/golden-thread/unlock/` and the marker beside it, `~/.claude/golden-thread/.unlock-unlock-on` (while either remains, `gt` treats unlock as on with no policy and keeps everything gated locked), then enrol again. Sealed credentials are lost with it — that is what sealed means; keep their originals in your store.
- **Turn it off:** `gt_unlock.py policy disable` (needs your factors) or `gt_settings.py set unlock off`. Off, every hook behaves exactly as before 0.20.0.
- **Check it:** `gt_unlock.py status`, `gt_unlock.py verify`, `gt_unlock.py audit -n 50`.

8. gt sandbox mode

`gt_settings.py set sandbox_mode on` (off by default). gt then configures **Claude Code's own sandbox and permission rules** so that the assistant's shell and file tools cannot touch the vault or gt's state, and the vault is reached only through gt's channels: a read-only MCP server and the write queue. It is independent of unlock — on together, unlock also gates the MCP server's reads (`gt:vault:read` , door `mcp_only`).

8.1 WHAT IT WRITES

Into `~/.claude/settings.json` , through `gt_sandbox.py` , which records every entry it adds (in `~/.claude/golden-thread/sandbox/state.json`) so that turning it off removes exactly those and nothing of yours. Every key is one the Claude Code documentation names (code.claude.com/docs/en/sandboxing, [/settings-reference](#), [/permissions](#), read 2026-10-03):

KEY	VALUE	WHY
<code>sandbox.enabled</code>	<code>true</code>	The OS sandbox around Bash, PowerShell and Mac tools and every process that starts
<code>sandbox.allowUnsandboxedCommands</code>	<code>false</code>	No "retry outside the sandbox" escape hatch: <code>dangerouslyDisableSandbox</code> is ignored
<code>sandbox.failIfUnavailable</code>	<code>true</code>	Claude Code refuses to start rather than silently run commands unsandboxed
<code>sandbox.filesystem.denyWrite</code>	the vault; <code>~/claude/golden-thread</code> (hooks, state, the unlock home and its <code>unlock-on</code> markers); the unlock socket's directory when it is outside that; <code>~/config/gt-lotr</code> and LOTR's store (<code>~/secrets</code>); <code>~/claude/plugins</code> ; <code>~/claude/settings.json</code> ; <code>~/claude/vault-config.json</code>	The shell cannot change the vault, gt's hooks and policies; the plugin files the author trusts, or the settings that turn this off
<code>sandbox.filesystem.denyRead</code>	the unlock home (sealed store, <code>totp.seed</code> , recovery codes, audit log); LOTR's home and store; every locked vault folder; and the whole vault while <code>sandbox_vault_reads</code> is <code>deny</code> (the default)	The shell cannot read the second factor, credential the vault
<code>sandbox.filesystem.allowWrite</code>	<code>~/gt-inbox</code>	The one writable place: <code>gt_write_queue.py</code> learns its requests in <code>~/gt-inbox/queue/</code>
<code>permissions.deny</code>	<code>Edit(//<path>/**)</code> for every write-denied path, <code>Read(//<path>/**)</code> for every read-denied path, and <code>Edit</code> on any <code>.claude/settings.json</code> / <code>.claude/settings.local.json</code> ; plus <code>Edit</code> on every file Claude Code, your login shell or launchd loads code or configuration from: <code>~/claude.json</code> , <code>~/claude/{settings.local.json, CLAUDE.md, agents, skills, commands, hooks, output-styles}</code> , any <code>.mcp.json</code> and any project <code>.claude/{agents,skills,commands,hooks,workflows}</code> , the shell rc files (<code>.zshrc</code> , <code>.zshenv</code> , <code>.zprofile</code> , <code>.zlogin</code> , <code>.bashrc</code> , <code>.bash_profile</code> , <code>.bash_login</code> , <code>.profile</code>), <code>~/Library/LaunchAgents</code> (macOS) and <code>~/config/systemd/user</code> / <code>~/config/autostart</code> (Linux, WSL2)	Claude's file tools (Read, Write, NotebookEdit, Glob) run outside the sandbox and follow permission rules instead; without the second group they could plant code that runs later outside every fence. Not all of <code>~/claude</code> is denied: Claude Code's auto-memory and plans live there
<code>permissions.disableBypassPermissionsMode</code>	<code>"disable"</code>	No <code>--dangerously-skip-permissions</code> / <code>bypass mode</code> while the fence is up; reset to its earlier value when the mode is turned off

`Write(...)` and `NotebookEdit(...)` path rules are not written: Claude Code checks file permissions against `Edit(path)` and `Read(path)` rules only, and an `Edit` deny covers every editing tool. Managed settings are never written; if they override

what gt asked for, `gt_sandbox.py check`, `gt_unlock.py verify` and `/gt:gt-doctor` say so.

Settings merge, so other files can loosen the fence. A repository's `.claude/settings.json` (loaded from the working directory) and `.claude/settings.local.json` (loaded from the git root) merge with your user settings, and can loosen the sandbox in that project: `sandbox.enabled false`, `allowUnsandboxedCommands true`, `excludedCommands`, `ignoreViolations`, Unix-socket and Mach-lookup entries, `enableWeakerNestedSandbox` / `enableWeakerNetworkIsolation`, `additionalDirectories`, or an `allowRead` / `allowWrite` / Edit or Read allow rule that re-opens a region gt denies (a narrower allow wins over a wider deny). `gt_sandbox.py check` reports every such key as a problem, naming the file — in your user settings too, so a user `excludedCommands` is no longer a pass. It **reports; it cannot prevent**. Claude Code ignores a repository's loosening keys only when the sandbox is **admin-required**:

`allowUnsandboxedCommands: false` set in **managed settings** or passed with `claude --settings <file>` (Claude Code v2.1.285 or later; user settings alone never make it admin-required). `gt_sandbox.py managed` prints that configuration for `managed-settings.json`, or saves it with `--out <file>` for `claude --settings`; gt never writes managed settings itself.

8.2 HOW THE VAULT IS REACHED

- **Reading:** gt's MCP server `gt-vault` (shipped in gt's plugin manifest; MCP servers run outside the sandbox) offers `vault_search` (index.md first, then pages scored like `/gt:gt-query`, current before expired before superseded), `vault_read` (size-capped at 64 KiB by default, 256 KiB at most, pageable) and `vault_list`. It is read-only, never returns a file outside the vault or inside a locked folder — even named exactly, and for `vault_search` even when a symlink in the vault points there (each result, and `index.md`, is resolved and checked the way `vault_read` checks a path) — and marks every result as untrusted data. With unlock on, each read needs `gt:vault:read`, served only to the session's registered `gt_vault_mcp.py` (checked from the kernel and by the installed file's real path, like LOTR's shim); a process from the assistant's shell is refused. By default `read_without_unlock` opens it like a LOTR read; set it false to need an unlock.
- **Writing:** `vault_queue_write` takes `gt_write_queue.py`'s ops and validation and queues the write; the broker decides it at once (apply, deduplicate, held, escalate, reject) and the result says which. From the shell, `gt_write_queue.py` cannot write the vault's queue folder, so it leaves the request in `~/ .gt-inbox/queue/`; `vault_queue_drain` (or `gt_broker.py drain` in a terminal) moves it into the queue. The inbox is writable by anything in the sandbox, so the broker treats each file as untrusted data. It opens the inbox folder once, refusing it if it (or `~/ .gt-inbox`) is a symlink, and opens each file **relative to that folder handle with no symlink following** (`O_NOFOLLOW`; a FIFO cannot block it), checks the **open file** — a regular file with exactly one link, within the size cap — and reads it once, bounded. Those bytes are the only ones it uses, for accepting and rejecting alike, and it removes the file through the same folder handle; it never re-opens an inbox path. So a sandboxed writer racing the broker cannot swap in a symlink, a hard link or another folder to have a file it may not read copied into the vault (the 2026-10-04 review demonstrated exactly that against 0.20.0's first build, with the TOTP seed; it is now a regression test, deterministic and live under the sandbox runtime). A request must name this vault and pass the queue's validation (paths inside the vault, no generated files, Sources or Core rules, a plain request id). The broker then **stamps** it: origin `inbox` (a body that said `farm` keeps `farm`, which is stricter) and session `unknown-inbox`, whatever the body claimed — a writer in the sandbox cannot speak for a session, so **no live session's claim lets an inbox request through** (Core rule 1), not even the claim of the session running the drain; it waits until the claim ends. It is then decided like any request — conflicts and `design.md` / `global-memory` go to you. Rejected inbox files are kept in `spool/broker/rejected/` up to 50; past that they are logged and deleted. An oversized file is never read and is left for you. `vault_queue_write` stamps its requests the same way, with the MCP server's own session id: a tool call cannot name another session to write through its claim. Native Windows has no sandbox, so there the inbox is not a boundary at all: the broker still refuses links and reparse points and checks that the open file is the one it listed, inside the inbox, but Claude's shell can open your files directly anyway.
- **Hooks and the broker** run outside the sandbox by Claude Code's design, so gt's enforcement keeps working.

8.3 WHAT IT STOPS, PER PLATFORM

PLATFORM	ENFORCED	AGAINST THE ASSISTANT'S SHELL
macOS	Seatbelt (OS) + permission rules	A boundary for the shell and every process it starts: writing the vault or gt's state, reading the unlock home, LOTR or the vault is refused by the kernel — as long as no settings file loosens it (8.1) and gt's own channels hold (the inbox broker reads only what the writer could; see 8.2)
Linux	bubblewrap (OS) + permission rules; needs bubblewrap and socat (gt refuses to turn the mode on without them, because Claude Code would then refuse to start)	Same as macOS
WSL2	bubblewrap, as Linux	Same as macOS
Native Windows	Permission rules only — Claude Code has no sandbox there	Friction, not a boundary: the file tools are refused, but a script the assistant runs can still open any file. <code>/gt:gt-doctor</code> and <code>gt_unlock.py verify</code> say "friction"

Verified on macOS (2026-10-03) with the sandbox runtime Claude Code builds on (`@anthropic-ai/sandbox-runtime 0.0.78`), using the lists gt generates and the working directory inside the vault: a write to the vault and a read of a vault page were both refused with `Operation not permitted` , a write to `~/gt-inbox` succeeded, a hard link to a vault file and a write through a symlink into the vault were refused, and `gt_write_queue.py` run inside it queued to the inbox and was applied by a drain outside. Claude Code's translation of `settings.json` into that runtime, and the file-tool permission rules, were checked by configuration (`claude sandbox status` reports the sandbox enabled and strict), not by a live Claude session. On 2026-10-04 the same runtime ran the review's inbox race for real (`tests/test_inbox_review2.py` , `H1LiveRace`): a writer inside it swapped an inbox file for a symlink to the TOTP seed about 8,000 times while the broker drained about 11,000 times outside — nothing reached the vault, where the pre-fix broker copied the seed in in the same test; the runtime itself refused a hard link to the seed.

8.4 WHAT IT DOES NOT STOP

- **Malware or any other program running as you.** The fence is around Claude's tools only; your terminal, editors, scheduled jobs and anything else you run are unaffected.
- **Claude Code's own processes outside the sandbox:** hooks, MCP servers (including LOTR and gt-vault, which can read the vault by design), plugin monitors, the status line. Commands you type at the `!` prompt usually run unsandboxed too.
- **What the vault MCP returns.** The assistant can still read the vault through it — that is the point; locked folders stay out of reach.
- **Network exfiltration of what the assistant can read.** gt pre-allows no domain; Claude Code's proxy asks before a sandboxed command reaches a new host (see the sandboxing page on domain fronting).
- **Native Windows** beyond the file tools (above).
- **A repository's own settings loosening the sandbox** (8.1) — unless you make it admin-required with managed settings or `claude --settings (gt_sandbox.py managed)` . `gt_sandbox.py check` reports it; it does not prevent it.

8.5 WHAT CHANGES FOR YOU

- Restart Claude Code after switching. Turning it **off** has to be done from a terminal: the sandbox write-protects `~/claude` from the assistant's shell, by design.
- gt's vault tools run from the assistant's shell (`gt_tasks` , `gt_lint` , `gt_log` , ...) cannot read or write the vault: the assistant uses the MCP tools, or you run them in a terminal. `sandbox_vault_reads allow` gives the shell read access back (writes stay denied).
- Test receipts, worker declarations and other state under `~/claude/golden-thread` cannot be written from the shell: run a release's test suite from a terminal.

- Git over SSH and `open / osascript` fail inside Claude Code's sandbox (its troubleshooting section has the workarounds); network hosts are asked for one at a time.
- Locking or unlocking a vault folder with `gt_lock.py` updates the locked-folder rules at once.

Check it any time: `gt_sandbox.py status` , `gt_sandbox.py check (drift)`, `gt_unlock.py verify` (rows `sandbox-*` ; run it from inside Claude Code and the `sandbox-live` row tries a real write to the vault and expects the OS to refuse it), `/gt:gt-doctor` (row `sandbox`).